



ΤΜΗΜΑ ΠΛΗΡΟΦΟΡΙΚΗΣ

DEPARTMENT OF INFORMATICS

Υπολογιστική Εργασία Ανάλυσης Εικόνας

2023-2024

Π20011 – Σωτήρης Χατζηκυριάκου

Π20025 – Αθηνά Αρμένη

Π20150 – Αλέξανδρος Βασίλειος Παναγόπουλος

Πίνακας περιεχομένων

Περιγραφή υπολογιστικής διαδικασίας	2
Προγραμματιστική υλοποίηση της εργασίας	6
Περιγραφή του κώδικα	7
Μέτρηση ακριβείας	12
Εξαγωγή χαρακτηριστικών.....	14
Παραδείγματα εκτέλεσης	16

Περιγραφή υπολογιστικής διαδικασίας

Ο αλγόριθμος που χρησιμοποιήσαμε ονομάζεται "Log-Based Hypergraph of Ranking Reference" και αποτελείται από πέντε βασικά βήματα. Προτού αναλυθούν αυτά τα βήματα, ας ορίσουμε ορισμένες έννοιες.

Αρχικά, ο αλγόριθμος λειτουργεί με ένα σύνολο εικόνων $C=\{o_1, o_2, \dots, o_n\}$, όπου κάθε o_i είναι μια εικόνα στο dataset και $|C|=n$ είναι ο συνολικός αριθμός των εικόνων. Στόχος του αλγορίθμου είναι για ένα συγκεκριμένο στόχο o_q να βρεθούν τα k πλησιέστερα αντικείμενα. Αυτό επιτυγχάνεται μέσω μιας ταξινομημένης λίστας $T_q=\{T_q(1), T_q(2), \dots, T_q(n)\}$, όπου $T_q(i) \in [N]$, όπου $T_q(i)$, $i \in [N]$ αντιστοιχεί στη θέση του αντικειμένου o_i στην ταξινομημένη λίστα του o_q . Για την ταξινόμηση αυτής της λίστας χρησιμοποιείται μια συνάρτηση ομοιότητας ρ . Έτσι, αν σε μια λίστα T_q , το o_i προηγείται του o_j , αυτό σημαίνει ότι $T_q(i) < T_q(j)$, και επομένως $\rho(q,i) \geq \rho(q,j)$. Στην περίπτωση μας, η αρχική συνάρτηση ομοιότητας ήταν το αντίστροφο της Ευκλείδειας απόστασης.

Τέλος, ορίζονται τα γειτονικά σύνολα $N(q,k)$, τα οποία περιέχουν τα k πλησιέστερα αντικείμενα ως προς το αντικείμενο-στόχο o_q .

Συνοψίζοντας, ο αλγόριθμος προσπαθεί να εντοπίσει τα πλησιέστερα αντικείμενα σε ένα dataset εικόνων μέσω μιας σειράς βημάτων: πρώτα, ταξινομεί τις εικόνες βάσει της ομοιότητάς τους με ένα συγκεκριμένο αντικείμενο-στόχο, και στη συνέχεια επιλέγει τα k πλησιέστερα σύμφωνα με την ταξινομημένη λίστα.

Βήματα

1. Η διαδικασία κανονικοποίησης της σειράς κατάταξης περιλαμβάνει την εφαρμογή ενός νέου μετρικού ομοιότητας $\rho_n(i,j)$ για κάθε λίστα T_i . Αυτός ο νέος μετρικός υπολογίζεται ως εξής: $\rho_n(i,j) = 2L - (t_i(j) - t_i(i))$

Σκοπός αυτής της διαδικασίας είναι να βελτιωθεί η συμμετρία των σχέσεων γειτονιάς $N(q,k)$. Η νέα μετρική λαμβάνει υπόψη τη σειρά κατάταξης των αντικειμένων σε κάθε λίστα, δίνοντας περισσότερη σημασία στην θέση τους. Η κανονικοποίηση αυτή μπορεί να βοηθήσει στη διασφάλιση ότι οι σχέσεις γειτνίασης είναι πιο ισορροπημένες και αξιόπιστες.

Μετά τον υπολογισμό του νέου μετρικού ομοιότητας, κάθε λίστα ταξινομείται ξανά βάσει αυτού του νέου μετρικού προκειμένου να είναι οι γειτονικές σχέσεις πιο αξιόπιστες και συμμετρικές.

2. Η κατασκευή ενός υπεργραφήματος αποτελεί μια γενίκευση ενός κανονικού γραφήματος, όπου οι ακμές αντιστοιχούν σε μη κενά υποσύνολα του συνόλου των κορυφών. Κάθε υπερακμή e_i αποτελείται από ένα σύνολο κορυφών $e_i = \{u_1, u_2, \dots, u_n\}$, και ορίζεται για κάθε εικόνα o_i με βάση το k γειτονικό σύνολο και τους αντίστοιχους γείτονες.

Στη δική μας υλοποίηση, κάθε υπερακμή είναι "κεντραρισμένη" σε μια εικόνα ή έναν κόμβο, όπου ο πρώτος κόμβος στο σύνολο κορυφών της υπερακμής είναι η v_i . Η συμμετοχή μιας εικόνας σε μια υπερακμή μπορεί να οριστεί με δυαδικό τρόπο, αλλά στην προκειμένη περίπτωση χρησιμοποιούμε πιθανοτική αναπαράσταση.

Το μέτρο συμμετοχής $r(e_i, u_j)$ της κορυφής u_j στην υπερακμή e_i υπολογίζεται ως εξής:

$$r(e_i, u_j) = \sum_{o_x \in N(i,k)} \sum_{o_j \in N(x,k)} w_p(i, x) \times w_p(x, j)$$

Όπου $w_p(i, x)$ είναι μια συνάρτηση που αναθέτει ένα βάρος στην o_x με βάση τη θέση της στην ταξινομημένη λίστα t_i . Η συνάρτηση αυτή υπολογίζεται ως:

$$w_p(i, x) = 1 - \log_k t_i(x)$$

Η συνάρτηση αυτή αναθέτει ένα μέγιστο βάρος ίσο με 1 στην πρώτη θέση της λίστας και μειώνει γρήγορα τα βάρη για τις πρώτες ταξινομημένες θέσεις.

Με βάση τις τιμές $r(e_i, u_j)$ ορίζουμε το incidence matrix $H(e_i, u_j)$ ως εξής:

$$H(e_i, u_j) = r(e_i, u_j)$$

Αν $u_j \notin e_i$, αλλιώς $H(e_i, u_j) = 0$.

Τέλος, ορίζεται ένα βάρος $w(e_i)$ για κάθε υπερακμή e_i , το οποίο αντιπροσωπεύει την αυτοπεποίθηση των σχέσεων που δημιουργούνται μεταξύ των κορυφών των υπερακμών. Αυτό υπολογίζεται ως εξής:

$$w(e_i) = \sum_{j \in N_h(i,k)} H(i, j)$$

όπου $N_h(i, k)$ είναι το γειτονικό σύνολο του o_i . Συνολικά, όσο μικρότερο το k και όσο πιο ποικίλο το σύνολο των κορυφών της υπερακμής, τόσο καλύτερη είναι η ποιότητα των υπερακμών και οι τιμές $H(e_i, u_j)$.

3. Η διαδικασία υπολογισμού της ομοιότητας υπερακμών ξεκινά με τον υπολογισμό του πίνακα ομοιότητας S μέσω του Hadamard product δύο matrices, S_h και S_w βάσει δύο υποθέσεων.

Η πρώτη υπόθεση θεωρεί ότι παρόμοιες εικόνες έχουν παρόμοιες ταξινομημένες λίστες και, συνεπώς, παρόμοιες υπερακμές. Επομένως, ο πίνακας ομοιότητας S_h υπολογίζεται ως το γινόμενο $H \times H^T$, όπου H είναι το incidence matrix.

Η δεύτερη υπόθεση προβλέπει ότι παρόμοιες εικόνες αναμένεται να αναφέρονται από τις ίδιες υπερακμές. Αυτό οδηγεί στον υπολογισμό ενός πίνακα ομοιότητας S_u ως $H^T \times H$.

Τελικά, ο πίνακας ομοιότητας S υπολογίζεται μέσω του Hadamard product των S_h και S_u , δηλαδή:

$$S = S_h \circ S_u$$

Με αυτόν τον τρόπο, η ομοιότητα μεταξύ υπερακμών λαμβάνει υπόψη τη συμμετρία των σχέσεων γειτονίας, καθώς και τη συχνότητα με την οποία οι εικόνες αναφέρονται από τις ίδιες υπερακμές.

4. Ο υπολογισμός του καρτεσιανού γινομένου μεταξύ των υπερακμών αποσκοπεί στην ανάγνωση των σχέσεων μεταξύ των κορυφών σε διαφορετικές υπερακμές. Αυτό επιτυγχάνεται μέσω της δημιουργίας ενός καρτεσιανού γινομένου, όπου κάθε ζεύγος κορυφών που ανήκουν σε διαφορετικές υπερακμές ορίζει μια σχέση σχετικότητας. Η διαδικασία αυτή παρέχει πληροφορίες για τη σχέση μεταξύ των κορυφών ανεξάρτητα από το ποιες υπερακμές ανήκουν.

Για κάθε ζεύγος κορυφών (u_i, u_j) που ανήκει στο καρτεσιανό γινόμενο των υπερακμών e_q και e_i , ορίζεται μια σχέση σχετικότητας. Στη συνέχεια, υπολογίζεται εκ νέου η συνάρτηση ρ με βάση το βάρος $w(e_q)$, που διατυπώνει την αυτοπεποίθηση της υπερακμής e_q που ορίζει τη συσχέτιση.

Ο βαθμός συμμετοχής των κορυφών u_i και u_j υπολογίζεται ως εξής:

$$\rho(e_q, u_i, u_j) = w(e_q) \times H(e_q, u_i) \times H(e_q, u_j)$$

Το μέτρο ομοιότητας του καρτεσιανού γινομένου υπολογίζεται μέσω ενός πίνακα C που λαμβάνει υπόψη τις σχέσεις σε όλες τις υπερακμές. Κάθε στοιχείο του πίνακα C υπολογίζεται ως εξής:

$$C(i, j) = \sum_{e_q \in E^{(u_i, u_j)}} \sum_{(u_i, u_j) \in e_q^2} \rho(u_i, u_j)$$

Με αυτόν τον τρόπο, ο υπολογισμός του καρτεσιανού γινομένου μεταξύ των υπερακμών επιτρέπει την ανάλυση των σχέσεων μεταξύ των κορυφών σε διαφορετικά περιβάλλοντα ή συνθήκες.

5. Ο υπολογισμός της ομοιότητας βάσει του κατασκευασμένου υπεργράφου συνδυάζει τις πληροφορίες που προέρχονται από το καρτεσιανό γινόμενο των υπερακμών και το προηγούμενα υπολογισμένο pairwise similarity matrix. Αυτό επιτυγχάνεται μέσω του πίνακα συγγένειας W , ο οποίος παρέχει μια συνολική αναπαράσταση της σχέσης μεταξύ των κορυφών.

Ο πίνακας W υπολογίζεται μέσω του Hadamard product (στοιχείο-προς-στοιχείο πολλαπλασιασμού) των πινάκων C και S . Αυτός ο πολλαπλασιασμός επιτρέπει τη συνδυασμένη χρήση των πληροφοριών από τις δύο πηγές για τον υπολογισμό του βαθμού συγγένειας.

Έπειτα, με βάση τον πίνακα W , μπορεί να εφαρμοστεί μια διαδικασία ranking για τη δημιουργία μιας νέας ταξινομημένης λίστας T . Η διαδικασία αυτή μπορεί να επαναληφθεί πολλές φορές, κάθε φορά ανανεώνοντας τον πίνακα W , μέχρι να επιτευχθεί η επιθυμητή σύγκλιση ή ένας άλλος κριτήριο τερματισμού.

Η διαδικασία ranking εφαρμόζεται για κάθε εικόνα q που ανήκει στο σύνολο των εικόνων. Το ακριβές πώς ορίζεται η διαδικασία αυτή εξαρτάται από το συγκεκριμένο πρόβλημα και τις απαιτήσεις της εφαρμογής. Συνήθως, μπορεί να χρησιμοποιηθούν αλγόριθμοι ranking όπως οι αλγόριθμοι PageRank ή άλλες προσαρμοσμένες προσεγγίσεις που λαμβάνουν υπόψη τις συγκεκριμένες ανάγκες της εφαρμογής.

Προγραμματιστική υλοποίηση της εργασίας

Η εργασία υλοποιήθηκε σε Python σε περιβάλλον του Visual Studio Code. Οι βιβλιοθήκες που χρησιμοποιήθηκαν για την πραγματοποίηση της ήταν οι `numpy`, `matplotlib.pyplot`, `torch` (μαζί με `torchvision.transforms`, `torchvision.models`), `Image`, `os` και `squeezenet1_1`. Στο αρχείο `"...".py` περιέχεται ο κώδικας της εφαρμογής ο οποίος είναι τεκμηριωμένος με σχόλια που θα εξηγήσουμε και με περισσότερη λεπτομέρεια παρακάτω. Ως σύνολο δεδομένων χρησιμοποιούμε το σύνολο δεδομένων `squeezenet1_1` (συγκεκριμένα ένα υποσύνολο του για να μειωθεί ο χρόνος εκτέλεσης). Το dataset αυτό περιέχεται στα datasets του `torchvision` και εγκαθίσταται μέσω της βιβλιοθήκης. Για την εξαγωγή των χαρακτηριστικών χρησιμοποιήσαμε την βιβλιοθήκη `matplotlib.pyplot`. Περισσότερες λεπτομέρειες για την συγκεκριμένη ενέργεια θα βρείτε στην περιγραφή του κώδικα. Ο κώδικας αρχίζει με την εισαγωγή των απαραίτητων βιβλιοθηκών και την ορισμό των βοηθητικών συναρτήσεων `reciprocal_rank_normalization` και `rank_lists`. Στη συνέχεια, φορτώνεται ένα προ-εκπαιδευμένο μοντέλο SqueezeNet από τη βιβλιοθήκη `torchvision`. Το μοντέλο αυτό χρησιμοποιείται για την εξαγωγή χαρακτηριστικών από τις εικόνες που παρέχονται στο σύνολο δεδομένων. Ορίζεται η συνάρτηση `extract_features`, η οποία χρησιμοποιεί το μοντέλο SqueezeNet για να εξάγει τα χαρακτηριστικά από μια εικόνα. Στη συνέχεια, ορίζεται μια λίστα με τα ονόματα των αρχείων εικόνων που θα χρησιμοποιηθούν για τη διαδικασία ταξινόμησης. Φορτώνονται οι εικόνες και εξάγονται τα χαρακτηριστικά τους, τα οποία χρησιμοποιούνται στη συνέχεια για τον υπολογισμό των ομοιοτήτων μεταξύ των εικόνων. Η ομοιότητα μεταξύ των εικόνων υπολογίζεται χρησιμοποιώντας το εσωτερικό γινόμενο των χαρακτηριστικών τους, παράγοντας έναν πίνακα ομοιότητας. Έπειτα, γίνεται η κανονικοποίηση του πίνακα ομοιότητας με τη μέθοδο `reciprocal_rank_normalization` και υπολογίζονται τα βάρη των υπερακμών. Ορίζεται ο πίνακας βαρών υπερακμών και υπολογίζεται ο πίνακας κατανομής πιθανοτήτων υπερακμών. Στη συνέχεια, γίνεται μια διαδικασία επαναληπτικής ταξινόμησης για τον υπολογισμό των τελικών ταξινομημένων λιστών. Τέλος, υπολογίζεται η ακρίβεια του συστήματος χρησιμοποιώντας μια υποθετική βάση δεδομένων αναφοράς και τα αποτελέσματα της ταξινόμησης που παράγει το σύστημα.

Περιγραφή του κώδικα

1. Υλοποίηση της Κανονικοποίηση Σειράς Κατάταξης (Rank Normalization)

Αρχικά, έχουμε τη συνάρτηση **reciprocal_rank_normalization(L, num_items)**:

```
def reciprocal_rank_normalization(L, num_items):
```

```
    tau_i = np.arange(num_items)
```

```
    tau_j = np.random.permutation(tau_i)
```

```
    rho_n = 2 * L - (tau_i + tau_j)
```

```
    rho_n[rho_n < 0] = 0
```

```
    return rho_n
```

Αυτή η συνάρτηση δέχεται δύο ορίσματα: το **L**, που αντιστοιχεί στον αριθμό των κορυφαίων θέσεων που θέλετε να εξετάσετε, και το **num_items**, που αναφέρεται στον συνολικό αριθμό των αντικειμένων που εξετάζονται. Η συνάρτηση χρησιμοποιείται για να υπολογίσει την κανονικοποιημένη βαθμολογία (ρο) για κάθε αντικείμενο.

1. Η γραμμή **tau_i = np.arange(num_items)** δημιουργεί έναν πίνακα **tau_i** που περιέχει τις διαδοχικές ακέραιες τιμές από 0 έως **num_items - 1**. Αυτός ο πίνακας αντιπροσωπεύει τις πραγματικές θέσεις των αντικειμένων στην αρχική κατάταξη.
2. Η γραμμή **tau_j = np.random.permutation(tau_i)** ανακατεύει τυχαία τις θέσεις του **tau_i** προκειμένου να προκύψει μια τυχαία κατάταξη, αντιπροσωπεύοντας τις νέες κατατάξεις.
3. Η γραμμή **rho_n = 2 * L - (tau_i + tau_j)** υπολογίζει την αντιστροφή ανακριτική κατάταξη για κάθε στοιχείο. Αυτό γίνεται αφαιρώντας τις θέσεις των στοιχείων από το διπλάσιο του **L** και προσθέτοντας 1. Αυτό το βήμα ουσιαστικά αντιστρέφει την κατάταξη και ταυτόχρονα εφαρμόζει την αντιστροφή ανακριτική κατάταξη.
4. Η γραμμή **rho_n[rho_n < 0] = 0** διορθώνει οποιεσδήποτε αρνητικές τιμές του **rho_n** και τις θέτει σε μηδέν, καθώς η κανονικοποίηση των κατατάξεων δεν πρέπει να οδηγεί σε αρνητικές τιμές.
5. Τέλος, η συνάρτηση επιστρέφει τον πίνακα **rho_n**, ο οποίος περιέχει τις κανονικοποιημένες κατατάξεις των στοιχείων.

Αυτός είναι ο τρόπος με τον οποίο η συνάρτηση **reciprocal_rank_normalization** παράγει τις κανονικοποιημένες κατατάξεις χρησιμοποιώντας την αντιστροφή ανακριτική κατάταξη.

2. Κατασκευή Υπεργράφου (Hypergraph Construction)

Ας δούμε τι κάνει κάθε μέρος του κώδικα:

```
# Convert feature matrix to a sparse incidence matrix
```

```
H = np.transpose(features)
```

```
# Compute similarity measure based on Cartesian product of hyperedges
```

```
num_vertices = H.shape[1] # Determine the number of vertices from the shape of H
```

```
S = np.zeros((num_vertices, num_vertices)) # Initialize similarity matrix S
```

```
# Compute pairwise similarities using Cartesian product of hyperedges
```

```
cartesian_product = []
```

```
for i in range(num_vertices):
```

```
    for j in range(num_vertices):
```

```
        # Calculate pairwise similarity relationship  $p(e_i, v_i, v_j)$ 
```

```
        similarity = np.dot(features[:, i], features[:, j])
```

```
S[i, j] = similarity
```

```
# Add Cartesian Product of Hyperedge Elements to the list
```

```
    cartesian_product.append((i+1, j+1)) # +1 to convert from zero-based to one-based indexing
```

1. Αρχικά, ο πίνακας χαρακτηριστικών **features** μετατρέπεται σε έναν αραιό πίνακα εισαγωγής **H** με τη χρήση της μετατόπισης των χαρακτηριστικών. Η μετατόπιση γίνεται γιατί η μεταβλητή **features** έχει διαστάσεις (**n_features, n_samples**) και θέλουμε το κάθε χαρακτηριστικό να αντιστοιχεί σε μια στήλη του **H**.
2. Στη συνέχεια, η μήτρα ομοιότητας **S** αρχικοποιείται ως ένας μηδενικός πίνακας με διαστάσεις (**num_vertices, num_vertices**), όπου **num_vertices** είναι ο αριθμός των κόμβων στον υπεργράφο.
3. Έπειτα, υπολογίζονται οι ομοιότητες μεταξύ των κόμβων χρησιμοποιώντας τον καρτεσιανό γινόμενο των υπερακμών. Για κάθε ζευγάρι κόμβων (**i, j**), η ομοιότητα υπολογίζεται ως το εσωτερικό γινόμενο των χαρακτηριστικών τους. Η ομοιότητα αυτή αποθηκεύεται στην αντίστοιχη θέση του πίνακα ομοιότητας **S**.
4. Επιπλέον, καταγράφονται τα στοιχεία του καρτεσιανού γινομένου των υπερακμών στη λίστα **cartesian_product**. Αυτό είναι χρήσιμο για περαιτέρω ανάλυση ή απεικόνιση του υπεργράφου.
Αυτός είναι ο τρόπος με τον οποίο ο κώδικας κατασκευάζει έναν υπεργράφο χρησιμοποιώντας τα χαρακτηριστικά των εικόνων και υπολογίζοντας τις ομοιότητες

3. Υπολογισμός Ομοιότητας Υπερακμών (Hyperedge Similarities)

Ας αναλύσουμε τι κάνει κάθε μέρος του κώδικα:

```
# Hyperedge Definition and Weight Assignment
probabilistic_incidence_matrix = np.zeros_like(S)
for i in range(num_items):
    for j in range(num_items):
        membership_measure = rho_n[i] * rho_n[j] # Use reciprocal rank normalization as the membership measure
        probabilistic_incidence_matrix[i, j] = membership_measure
# Hyperedge Weight Calculation
hyperedge_weights_calculated = np.sum(probabilistic_incidence_matrix, axis=1)
# Define W based on C and S (element-wise multiplication)
C = probabilistic_incidence_matrix # Placeholder for C
W = C * S # Calculate W based on element-wise multiplication of C and S
```

1. Αρχικά, δημιουργείται ένας μηδενικός πίνακας **probabilistic_incidence_matrix** με τις ίδιες διαστάσεις με τον πίνακα ομοιότητας **S**. Αυτός ο πίνακας θα χρησιμοποιηθεί για τον υπολογισμό των βαρών των υπερακμών.
2. Στη συνέχεια, υπολογίζονται τα βάρη των υπερακμών μέσω της **reciprocal_rank_normalization** για κάθε δύο εικόνες **i** και **j**. Αυτή η μέτρηση μέλους χρησιμοποιεί την αντιστροφή ανακριτική κατάταξη ως μέτρο προσήμου και πολλαπλασιάζει τις τιμές για την υπολογιστική απλότητα.
3. Τα υπολογισμένα βάρη των υπερακμών αθροίζονται για κάθε υπερακμή, προκειμένου να υπολογιστούν τα συνολικά βάρη για κάθε υπερακμή.
4. Τέλος, το βάρος των υπερακμών **W** ορίζεται μέσω του γινομένου του πίνακα **C** και του πίνακα ομοιότητας **S**. Αυτό το κάνει με την στοιχειοθετική πολλαπλασιασμένη των δύο πινάκων. Αυτό το βάρος χρησιμοποιείται στον υπεργράφο για να αντιπροσωπεύσει τις σχέσεις ομοιότητας μεταξύ των κόμβων.

4. Υπολογισμός Καρτεσιανού Γινομένου μεταξύ των στοιχείων των Υπερακμών (Cartesian Product of Hyperedge Elements)

Τα κομμάτια του κώδικα που υπολογίζουν το Καρτεσιανό Γινόμενο των Στοιχείων των Υπερακμών βρίσκονται στον παρακάτω κώδικα:

```
# Compute pairwise similarities using Cartesian product of hyperedges
cartesian_product = []
for i in range(num_vertices):
    for j in range(num_vertices):
        # Calculate pairwise similarity relationship p(eq, vi, vj)
        similarity = np.dot(features[:, i], features[:, j])
        S[i, j] = similarity
        # Add Cartesian Product of Hyperedge Elements to the list
        cartesian_product.append((i+1, j+1)) # +1 to convert from zero-based to one-based indexing
```

Ας δούμε πώς λειτουργεί κάθε μέρος του κώδικα:

1. Ο διπλός βρόγχος **for** χρησιμοποιείται για να δημιουργήσει ένα Καρτεσιανό Γινόμενο των στοιχείων των υπερακμών. Κάθε στοιχείο του πίνακα **features** αντιστοιχεί σε έναν κόμβο του υπεργράφου.
2. Μέσα στον εσωτερικό βρόγχο, υπολογίζεται η ομοιότητα μεταξύ των δύο κόμβων **i** και **j**. Αυτή η ομοιότητα υπολογίζεται ως το εσωτερικό γινόμενο των χαρακτηριστικών τους. Το αποτέλεσμα αυτής της υπολογιστικής διαδικασίας αντιστοιχεί στην ομοιότητα μεταξύ των δύο κόμβων.
3. Η ομοιότητα αυτή αποθηκεύεται στην αντίστοιχη θέση του πίνακα ομοιότητας **S**, δηλαδή **S[i, j]**.
4. Τέλος, οι δείκτες των κόμβων **i** και **j** προστίθενται στη λίστα **cartesian_product**. Αυτό κάνει την αναπαράσταση του Καρτεσιανού Γινομένου των Στοιχείων των Υπερακμών, όπου κάθε ζευγάρι **(i, j)** αντιπροσωπεύει μια σύνδεση μεταξύ δύο κόμβων στον υπεργράφο.

Αυτός είναι ο τρόπος με τον οποίο ο κώδικας υπολογίζει το Καρτεσιανό Γινόμενο των Στοιχείων των Υπερακμών μεταξύ των κόμβων του υπεργράφου.

5. Υπολογισμός Ομοιότητας βάσει του κατασκευασμένου Υπεργράφου (Hypergraph – Based Similarity)

Τα κομμάτια του κώδικα που χρησιμοποιούνται για τον υπολογισμό της ομοιότητας βασισμένης σε υπεργράφο (Hypergraph-Based Similarity) βρίσκονται στον παρακάτω κώδικα:

```
# Reciprocal Rank Normalization
L = 4 # Top-L positions
num_items = len(image_list)
rho_n = reciprocal_rank_normalization(L, num_items)
# Hyperedge Definition and Weight Assignment
probabilistic_incidence_matrix = np.zeros_like(S)
for i in range(num_items):
    for j in range(num_items):
        membership_measure = rho_n[i] * rho_n[j] # Use reciprocal rank normalization as the membership measure
        probabilistic_incidence_matrix[i, j] = membership_measure
# Hyperedge Weight Calculation
hyperedge_weights_calculated = np.sum(probabilistic_incidence_matrix, axis=1)
# Define W based on C and S (element-wise multiplication)
C = probabilistic_incidence_matrix # Placeholder for C
W = C * S # Calculate W based on element-wise multiplication of C and S
```

Ας δούμε τι κάνει κάθε μέρος του κώδικα:

1. Αρχικά, ορίζουμε την παράμετρο **L**, η οποία καθορίζει τον αριθμό των κορυφαίων θέσεων για τις οποίες θα γίνει ο υπολογισμός του βάρους.
2. Υπολογίζουμε τον πίνακα **rho_n** με τη χρήση της συνάρτησης **reciprocal_rank_normalization**. Αυτή η συνάρτηση χρησιμοποιεί την αντιστροφή κανονικοποίησης της κατάταξης (Reciprocal Rank Normalization) ως μέτρο προσήμου για την ομοιότητα.
3. Στη συνέχεια, δημιουργούμε τον πίνακα πιθανοτικής εισαγωγής **probabilistic_incidence_matrix**. Κάθε στοιχείο αυτού του πίνακα υπολογίζεται ως το γινόμενο της κανονικοποιημένης αντίστροφης κατάταξης των δύο εικόνων **i** και **j**.
4. Τα υπολογισμένα βάρη των υπερακμών υπολογίζονται με το άθροισμα κάθε γραμμής του πίνακα πιθανοτικής εισαγωγής.
5. Τέλος, ο Πίνακας Βάρων **W** ορίζεται ως το γινόμενο των πινάκων **C** και **S** με στοιχειοθετική πολλαπλασιασμένη. Αυτός ο πίνακας παριστά τα βάρη των υπερακμών στον υπεργράφο, λαμβάνοντας υπόψη τις ομοιότητες μεταξύ των κόμβων.

Μέτρηση ακρίβειας

Συστηματική διαδικασία για την μέτρηση της ακρίβειας

Η διαδικασία για τη μέτρηση της ακρίβειας του συγκεκριμένου αλγορίθμου είναι ως εξής:

```
# Υποθετικό σύνολο δεδομένων αναφοράς
ground_truth = {
    'I1.jpg': ['I4.jpg', 'I3.jpg'],
    'I2.jpg': [],
    'I3.jpg': ['I4.jpg', 'I1.jpg'],
    'I4.jpg': ['I1.jpg', 'I3.jpg']
}

# Υποθετική εφαρμογή του αλγορίθμου στο σύνολο δεδομένων ελέγχου
results = {
    'I1.jpg': ['I2.jpg', 'I3.jpg', 'I4.jpg'],
    'I2.jpg': ['I3.jpg', 'I4.jpg', 'I1.jpg'],
    'I3.jpg': ['I4.jpg', 'I2.jpg', 'I1.jpg'],
    'I4.jpg': ['I1.jpg', 'I2.jpg', 'I3.jpg']
}

# Υπολογισμός ακρίβειας
total_queries = len(ground_truth)
correct_results = 0
total_relevant_images = 0
returned_relevant_images = 0
for query_image, related_images in ground_truth.items():
    if query_image in results:
        returned_images = results[query_image]
        total_relevant_images += len(related_images)
        for related_image in related_images:
            if related_image in returned_images:
                correct_results += 1
                returned_relevant_images += 1
accuracy = correct_results / total_queries
precision = returned_relevant_images / (len(results) * len(ground_truth))
recall = returned_relevant_images / total_relevant_images
print(f"Ακρίβεια: {accuracy}")
print(f"Προσέγγιση: {precision}")
print(f"Ανάκληση: {recall}")
```

Ας δούμε πώς λειτουργεί κάθε μέρος του κώδικα:

1. Υποθετικό Σύνολο Δεδομένων Αναφοράς (Ground Truth): Πρώτα, ορίζουμε ένα υποθετικό σύνολο δεδομένων αναφοράς (ground_truth) που περιέχει πληροφορίες για την πραγματική σχέση μεταξύ των εικόνων. Για κάθε εικόνα ερωτήματος (query image), έχουμε μια λίστα με τις πραγματικές σχετικές εικόνες.

2. Αποτελέσματα Αλγορίθμου (Results): Στη συνέχεια, ορίζουμε ένα άλλο υποθετικό σύνολο δεδομένων (results) που περιέχει τα αποτελέσματα του αλγορίθμου αναζήτησης για κάθε εικόνα ερωτήματος.
3. Υπολογισμός Ακρίβειας: Έπειτα, υπολογίζουμε την ακρίβεια του αλγορίθμου. Πηγαίνουμε μέσα από το υποθετικό σύνολο δεδομένων αναφοράς και για κάθε εικόνα ερώτησης ελέγχουμε αν οι εικόνες που επιστρέφονται από τον αλγόριθμο είναι πραγματικά σχετικές με την ερώτηση. Αυτό γίνεται μετρώντας πόσες από τις επιστρεφόμενες εικόνες βρίσκονται στο σύνολο των πραγματικά σχετικών εικόνων.
4. Υπολογισμός Προσέγγισης και Ανάκλησης: Τέλος, υπολογίζουμε την προσέγγιση και την ανάκληση του αλγορίθμου. Η προσέγγιση μας δίνει μια ιδέα για το πόσες από τις επιστρεφόμενες εικόνες είναι πραγματικά σχετικές, ενώ η ανάκληση μας δίνει μια ιδέα για το πόσες από τις πραγματικά σχετικές εικόνες επιστρέφονται από τον αλγόριθμο.

Με αυτόν τον τρόπο, μπορούμε να αξιολογήσουμε την απόδοση του αλγορίθμου μας σε ένα υποθετικό σενάριο, παρόλο που το πραγματικό σενάριο μπορεί να διαφέρει ανάλογα με τα δεδομένα και τις πρακτικές εφαρμογές.

Εξαγωγή χαρακτηριστικών

Για την εξαγωγή των δεδομένων:

1. **Φόρτωση Προεκπαιδευμένου Μοντέλου:** Η γραμμή `model = squeezeNet1_1(pretrained=True)` φορτώνει το προεκπαιδευμένο μοντέλο SqueezeNet.
2. **Προεπεξεργασία Εικόνων:** Η μεταβλητή `preprocess` περιέχει ένα σύνολο μετασχηματισμών που εφαρμόζονται στις εικόνες πριν εισαχθούν στο μοντέλο.
3. **Εξαγωγή Χαρακτηριστικών Εικόνων:** Η συνάρτηση `extract_features` δέχεται μια εικόνα ως είσοδο, εφαρμόζει τους μετασχηματισμούς προεπεξεργασίας και στη συνέχεια εξάγει τα χαρακτηριστικά της εικόνας χρησιμοποιώντας το προεκπαιδευμένο μοντέλο SqueezeNet.
4. **Δημιουργία Αραιού Πίνακα Εμφάνισης:** Ο αραιός πίνακας εμφάνισης `H` δημιουργείται από τα χαρακτηριστικά των εικόνων.
5. **Υπολογισμός Ομοιότητας:** Ο πίνακας ομοιότητας `S` υπολογίζεται χρησιμοποιώντας τον αραιό πίνακα εμφάνισης `H`.
6. **Κανονικοποίηση Αντίστροφης Τάξης:** Η συνάρτηση `reciprocal_rank_normalization` υπολογίζει την αντίστροφη τάξη για να κανονικοποιήσει τα βάρη των υπερσυνδέσεων.
7. **Υπολογισμός Υπερσυνδέσεων και Βαρών:** Υπολογίζονται οι υπερσυνδέσεις και τα βάρη τους χρησιμοποιώντας τον πίνακα ομοιότητας και την αντίστροφη τάξη.
8. **Δημιουργία Αφινότητας Πίνακα:** Ο πίνακας αφινότητας `W` υπολογίζεται από τον πολλαπλασιασμό του πίνακα υπερσυνδέσεων και του πίνακα ομοιότητας.

Τώρα, όσον αφορά τη δημιουργία του γραφήματος:

1. **Δημιουργία Υπολογιστικού Γραφήματος:** Το γράφημα περιλαμβάνει εικόνες των αντικειμένων, πίνακες με δεδομένα (όπως ο πίνακας ομοιότητας `S` και ο πίνακας αφινότητας `W`), καθώς και τα αποτελέσματα της ανάλυσης.
2. **Συντονισμένη Προβολή Δεδομένων:** Η συντονισμένη προβολή παρουσιάζει τα δεδομένα (εικόνες, πίνακες, αναφορές) σε μια οπτικά ευανάγνωστη μορφή.
3. **Προσθήκη Αναφορών και Δεδομένων:** Οι αναφορές και τα δεδομένα προστίθενται στα αντίστοιχα μέρη του γραφήματος για να βοηθήσουν στην ερμηνεία των αποτελεσμάτων.

4. **Προσθήκη Κειμένου και Επεξήγησης:** Το κείμενο προστίθεται για να επεξηγήσει τα βήματα του αλγορίθμου και την ερμηνεία των δεδομένων

Πιο συγκεκριμένα για το `plt`:

Το **plt** αναφέρεται στο module **matplotlib.pyplot**, το οποίο είναι ένα πολύ δημοφιλές εργαλείο στην Python για την απεικόνιση δεδομένων και τη δημιουργία γραφημάτων. Ο κώδικας που χρησιμοποιεί το **plt** στο παραπάνω παράδειγμα χρησιμοποιείται για τη δημιουργία ενός subplot grid που περιέχει διάφορα γραφήματα και εικόνες. Ας δούμε τι κάνει ο κώδικας που χρησιμοποιεί το **plt**:

1. **Δημιουργία Subplot Grid:** Η εντολή **fig, axes = plt.subplots(num_annotation_rows, num_cols_needed, figsize=(15, 12))** δημιουργεί ένα subplot grid με δεδομένο αριθμό γραφημάτων και εικόνων, καθώς και μεγέθη για το πλάτος και το ύψος του.
2. **Προσθήκη Εικόνων στο Subplot Grid:** Οι εικόνες προστίθενται στα subplot χρησιμοποιώντας τη μέθοδο **imshow()**.
3. **Προσθήκη Κειμένου στα Subplots:** Το κείμενο προστίθεται στα subplots χρησιμοποιώντας τη μέθοδο **text()**.
4. **Ρύθμιση Περιεχομένων Subplots:** Ρυθμίσεις για την απόκρυψη των αξόνων (**axis('off')**) και τον διαχωρισμό των subplots (**subplots_adjust()**).
5. **Εμφάνιση του Σχήματος:** Τέλος, η εντολή **plt.show()** εμφανίζει το σχήμα με όλα τα subplot και τις εικόνες.

Συνολικά, το **plt** χρησιμοποιείται για να οργανώσει και να εμφανίσει διάφορα γραφήματα και εικόνες με έναν εύκολο τρόπο.

Παραδείγματα εκτέλεσης

Παράδειγμα 1^ο :

Query Image: Image 4



Accuracy:1.5
Precision:0.375
Recall:1.0

Image 2



Image 3



Image 1



Pairwise Similarity Matrix (S):
[[288.86749268 150.4888916 336.04586792 361.97061157]
[150.4888916 99.68222046 188.0222168 206.38336182]
[336.04586792 188.0222168 430.50762939 459.56280518]
[361.97061157 206.38336182 459.56280518 494.8788147]]

Probabilistic Incidence Matrix:
[[49. 35. 42. 14.]
[35. 25. 30. 10.]
[42. 30. 36. 12.]
[14. 10. 12. 4.]]

Hyperedge Weights (Calculated):
[140. 100. 120. 40.]

Affinity Matrix (W):
[[14154.50714111 5267.11120605 14113.92645264 5067.58856201]
[5267.11120605 2492.05551147 5640.66650391 2063.83361816]
[14113.92645264 5640.66650391 15498.2746582 5514.75366211]
[5067.58856201 2063.83361816 5514.75366211 1979.51525879]]

Cartesian Product of Hyperedge Elements:
(1, 1)
(1, 2)
(1, 3)
(1, 4)
(2, 1)
(2, 2)
(2, 3)
(2, 4)
(3, 1)
(3, 2)
(3, 3)
(3, 4)
(4, 1)
(4, 2)
(4, 3)
(4, 4)

Παράδειγμα 2^ο :

Query Image: Image 1



Accuracy:2.25
Precision:0.5625
Recall:1.0

Image 2



Image 3



Image 4



Pairwise Similarity Matrix (S):
[[294.33782959 235.32513428 314.93093872 271.55380249]
[235.32513428 206.08972168 241.66583252 212.80096436]
[314.93093872 241.66583252 343.2590332 292.85977173]
[271.55380249 212.80096436 292.85977173 251.83677673]]

Probabilistic Incidence Matrix:
[[25. 25. 25. 25.]
[25. 25. 25. 25.]
[25. 25. 25. 25.]
[25. 25. 25. 25.]]

Hyperedge Weights (Calculated):
[100. 100. 100. 100.]

Affinity Matrix (W):
[[7358.44573975 5883.12835693 7873.27346802 6788.84506226]
[5883.12835693 5152.24304199 6041.64581299 5320.02410889]
[7873.27346802 6041.64581299 8581.47583008 7321.49429321]
[6788.84506226 5320.02410889 7321.49429321 6295.91941833]]

Cartesian Product of Hyperedge Elements:
(1, 1)
(1, 2)
(1, 3)
(1, 4)
(2, 1)
(2, 2)
(2, 3)
(2, 4)
(3, 1)
(3, 2)
(3, 3)
(3, 4)
(4, 1)
(4, 2)
(4, 3)
(4, 4)